

# AI시킴 앱/웹사이트 개발 집중팩

AI시킴 고객 전달용 최종 산출물

이 PDF는 실제 발송/판매용 원본을 기준으로 제작된 버전입니다.

# AI시킴 앱/웹사이트 개발 집중팩

개발 요청, 기능 정의, 구조 제안, 최종 검수까지 한 흐름으로 정리한 실전 작업지시서

---

## 이 팩이 특히 잘 맞는 사람

- 개발자에게 무엇을 어떻게 요청해야 할지 막막한 사람
- 기능 정의와 요구사항 정리를 더 명확하게 하고 싶은 사람
- 개발 결과물을 배포 전에 더 꼼꼼하게 검수하고 싶은 사람

## 먼저 이렇게 사용하세요

1. 기능 정의가 막힌다면 '기능 요구사항을 처음 정리하고 싶을 때'부터 시작합니다.
2. 개발자 전달 문장이 약하다면 '개발자에게 전달할 작업 지시문을 만들고 싶을 때'를 이어서 사용합니다.
3. 결과물을 내보내기 전에는 '개발 결과물을 최종 검수하고 싶을 때'로 마무리합니다.

## 이 팩은 왜 따로 존재하나요?

이 패키지는 특정 분야에서 바로 써먹을 수 있는 작업지시서만 빠르게 꺼내 쓰고 싶은 사용자를 위해 분리한 집중팩입니다.

즉, 안 쓸 분야까지 같이 사는 느낌보다 '내가 지금 필요한 문제를 바로 해결하는 느낌'이 먼저 오도록 구성합니다.

## 핵심 작업지시서

### 기능 요구사항을 처음 정리하고 싶을 때

#### 한 줄 설명

비개발자도 기능 아이디어를 개발자에게 전달 가능한 수준으로 정리하게 돕는 요구사항 정리형 작업지시서입니다.

#### 이런 상황에서 특히 잘 맞습니다

- 만들고 싶은 기능은 있는데 설명이 안 잡힐 때

- 기능 정의를 문서로 정리해야 할 때
- MVP 범위와 확장 범위를 분리해야 할 때

## 이 작업지시서로 기대할 수 있는 결과

- 문제 정의
- 사용자 흐름
- 핵심 요소
- MVP 범위
- 확장 가능 요소
- 개발자 전달용 요약

## 언제 쓰는지

- 개발 요청 전 요구사항 정리 단계
- 기획 초안에서 구현 범위를 좁혀야 할 때

## 바로 복사할 본문

지금 저는 [서비스 또는 화면]에 들어갈 [기능]을 정의하고 싶습니다.  
 이 기능의 목적은 [사용자 문제 해결 / 운영 효율 / 전환율 개선 / 사용성 개선]입니다.  
 현재 가장 중요하게 보고 싶은 것은 [사용자 흐름 / 핵심 요소 / MVP 범위 / 개발자 전달용 요구사항]입니다.

먼저 이 기능을 정의하기 전에 꼭 확인해야 할 질문이 있다면 먼저 물어봐 주세요.  
 그 다음 아래 형식으로 기능 정의를 정리해 주세요.

1. 이 기능이 해결하려는 문제
2. 사용자가 이 기능을 쓰는 흐름
3. 꼭 필요한 핵심 요소
4. MVP에서 우선 구현할 범위
5. 추후 확장 가능 요소
6. 개발자에게 전달할 수 있는 요구사항 요약

답변은 비개발자도 이해할 수 있게 설명하되, 개발자가 읽어도 바로 작업을 시작할 수 있을 정도로 구체적으로 써주세요.  
 애매한 표현보다 실제 화면이나 동작이 떠오르는 설명을 우선해 주세요.

## 바꿔야 할 부분

- '[서비스 또는 화면]': 예) AI시킴 결과 페이지, 결제 페이지
- '[기능]': 예) 이메일 수집 폼, 결제 버튼, 복사 기능
- '[사용자 문제 해결 / 운영 효율 / 전환율 개선 / 사용성 개선]': 기능 목적
- '[사용자 흐름 / 핵심 요소 / MVP 범위 / 개발자 전달용 요구사항]': 우선 포인트

## 후속 질문

- 위 기능 정의를 개발자 전달용 문서 형식으로 다시 정리해줘.
- 사용자 흐름을 화면 단위로 더 세분화해서 보여줘.

## 수정 요청

- 너무 추상적인 표현은 줄이고 실제 화면 요소 기준으로 써줘.
- 기술 설명보다 먼저 사용자 문제와 목적이 더 잘 보이게 다시 정리해줘.

## 검수 요청

- 이 기능 정의에서 빠진 핵심 요소가 있는지 점검해줘.
- 개발자가 읽었을 때 애매하게 느낄 부분이 있는지 검수해줘.

## 자주 하는 실수

- 기능 이름만 적고 목적을 비워둔다
- MVP 범위와 확장 범위를 섞어서 말한다
- 개발자에게 전달할 언어로 정리하지 못한다

## 이 순서에서 쓰면 좋습니다

1. 이 작업지시서로 기능 정의를 만든다
2. 개발자 전달용 작업지시문 생성형으로 바꾼다
3. 구현안이 나온 뒤 개선 요청형이나 검수형으로 이어간다

## 사용 팁

기능 정의는 "멋있어 보이는 설명"보다 "누가 봐도 오해가 적은 설명"이 중요합니다.

목적과 MVP 범위가 선명하면 이후 개발 흐름이 훨씬 쉬워집니다.

---

## 사용자 흐름을 화면 단위로 정리하고 싶을 때

### 한 줄 설명

서비스 기능이나 화면 아이디어를 페이지/단계 단위로 정리해, 실제 UX 흐름이 보이도록 만드는 작업지시서입니다.

## 이런 상황에서 특히 잘 맞습니다

- 기능은 있는데 사용 흐름이 안 잡힐 때
- 화면이 여러 개 연결되는데 어디서 뭘 해야 할지 정리 안 될 때
- 기획자/개발자/디자이너가 같은 흐름을 보게 만들고 싶을 때

## 이 작업지시서로 기대할 수 있는 결과

- 단계별 사용자 흐름
- 화면별 역할
- 전환 지점과 이탈 지점 정리

## 언제 쓰는지

- 기능 정의 이후 UX 흐름 정리 단계
- 화면 설계 전/후 보완 단계

## 바로 복사할 본문

지금 저는 [서비스/기능]의 사용자 흐름을 화면 단위로 정리하고 싶습니다.  
이 흐름의 목적은 [신청 / 진단 / 구매 / 탐색 / 결과 확인]입니다.  
가장 중요한 것은 사용자가 어느 화면에서 무엇을 하고, 다음 화면으로 왜 이동하는지가 분명히 보이는 것입니다.

아래 기준으로 사용자 흐름을 정리해 주세요.

1. 시작 화면
2. 다음 단계 화면들
3. 각 화면의 목적
4. 사용자가 해야 하는 행동
5. 다음 화면으로 이동하는 이유
6. 이탈 가능성이 큰 지점

답변은 화면 단위로 끊어서 정리해 주세요.

가능하면 비개발자도 이해할 수 있게 설명하되, 기획/개발 문서로 써도 될 정도로 분명하게 정리해 주세요.

## 바꿔야 할 부분

- `[서비스/기능]`
- `[신청 / 진단 / 구매 / 탐색 / 결과 확인]`

## 후속 질문

- 위 흐름을 모바일 사용자 기준으로 더 단순하게 다시 정리해줘.
- 각 화면의 핵심 CTA까지 같이 표시해줘.

## 수정 요청

- 화면 간 이동 이유가 더 잘 보이게 다시 써줘.
- 이탈 가능성이 높은 구간을 더 분명히 표시해줘.

## 검수 요청

- 이 흐름만 봐도 사용자가 어디서 무엇을 하는지 충분히 보이는지 검수해줘.
- 빠진 핵심 화면이 있는지 점검해줘.

## 사용 팁

사용자 흐름은 화면 개수보다 “왜 다음으로 이동하는지”가 선명해야 실제 설계가 쉬워집니다.

---

# 개발자에게 전달할 작업 지시문을 만들고 싶을 때

## 한 줄 설명

기획자나 운영자가 떠올린 요구사항을 개발자가 오해 없이 착수할 수 있는 작업 지시문으로 바꾸는 작업지시서입니다.

## 이런 상황에서 특히 잘 맞습니다

- 슬랙, 노션, 티켓에 올릴 개발 요청이 필요할 때
- 기능은 정했지만 전달 문장이 막막할 때
- 비개발자가 개발자와 협업할 때

## 이 작업지시서로 기대할 수 있는 결과

- 작업 목적
- 해결하려는 문제
- 작업 범위
- 핵심 요구사항
- 제외 범위
- 완료 후 기대 상태

## 언제 쓰는지

- 개발 요청 직전

- 문서화가 필요한 협업 단계

## 바로 복사할 본문

지금 저는 [서비스 / 화면 / 기능]에 대해 개발자에게 작업 요청을 해야 합니다.  
현재 제가 이해하고 있는 목적은 [사용자 문제 / 전환율 개선 / 운영 효율 / 사용성 개선]입니다.  
특히 이번 요청에서 가장 중요한 것은 [핵심 동작 / 사용자 흐름 / 구현 우선순위 / 완료 기준]입니다.

먼저 개발자가 이해하기 위해 부족한 정보가 있다면 먼저 질문해 주세요.  
그 다음 아래 형식으로 개발자 전달용 작업 지시문을 작성해 주세요.

1. 작업 목적
2. 해결하려는 문제
3. 작업 범위
4. 꼭 필요한 요구사항
5. 제외해도 되는 범위
6. 완료 후 기대 상태

문장은 애매하지 않게 써주세요.  
비개발자가 읽어도 이해할 수 있어야 하지만, 개발자가 읽었을 때 바로 착수 가능한 수준으로 정리해 주세요.  
모호한 표현보다 실제 동작이 떠오르는 표현을 우선해 주세요.

## 바뀌야 할 부분

- `[서비스 / 화면 / 기능]`: 예) AI시킴 이메일 수집 폼
- `[사용자 문제 / 전환율 개선 / 운영 효율 / 사용성 개선]`: 실제 목적
- `[핵심 동작 / 사용자 흐름 / 구현 우선순위 / 완료 기준]`: 우선 포인트

## 후속 질문

- 위 작업 지시문을 티켓 등록용 형태로 더 짧게 다시 써줘.
- 완료 기준이 더 명확하게 보이도록 다시 정리해줘.

## 수정 요청

- 너무 추상적인 표현은 줄이고 실제 동작 기준으로 써줘.
- 개발 범위와 제외 범위를 더 선명하게 나눠줘.

## 검수 요청

- 개발자가 읽었을 때 애매하게 느낄 부분이 있는지 검수해줘.
- 빠진 요구사항이 있다면 최소한으로 보완해줘.

## 자주 하는 실수

- “이거 만들어줘” 식으로 목적 없이 요청한다

- 완료 상태를 정의하지 않는다
- 제외 범위를 안 적어서 요청 범위가 계속 커진다

## 이 순서에서 쓰면 좋습니다

1. 기능 정의형으로 문제와 흐름을 정리한다
2. 이 작업지시서로 전달 문서를 만든다
3. 구현안이 나온 뒤 개선 요청형으로 이어간다

## 사용 팁

개발자 전달 문서는 “예쁘게 써진 설명”보다 “오해 없이 일 시작할 수 있는 문장”이 더 중요합니다.

---

---

## 버그 상황을 더 정확하게 설명하고 싶을 때

### 한 줄 설명

애매한 버그 설명을 개발자나 AI가 바로 이해할 수 있는 재현 가능한 설명으로 바꾸는 작업지시서입니다.

### 이런 상황에서 특히 잘 맞습니다

- “이상하게 안 돼요” 수준에서 더 설명이 안 될 때
- 재현 조건과 기대 결과를 정리해야 할 때
- 개발자와 커뮤니케이션이 자꾸 엇갈릴 때

### 이 작업지시서로 기대할 수 있는 결과

- 버그 요약
- 재현 단계
- 실제 결과와 기대 결과 구분
- 추가 확인 포인트

### 언제 쓰는지

- 버그 보고 직전
- 개발자 전달용 이슈 정리 단계



## 바로 복사할 본문

지금 저는 [버그/이상 현상]을 개발자에게 더 정확하게 설명하고 싶습니다.

현재 상태에서는 상황을 막연하게만 설명하고 있어서, 재현 가능하고 오해 없는 방식으로 정리하고 싶습니다.

아래 기준으로 버그 설명 문서를 정리해 주세요.

1. 어떤 화면/기능에서 발생하는지
2. 어떻게 하면 같은 현상이 재현되는지
3. 실제로 나타난 결과
4. 원래 기대했던 정상 결과
5. 추가로 확인하면 좋은 정보

답변은 아래 순서로 주세요.

1. 버그 한 줄 요약
2. 재현 단계
3. 실제 결과
4. 기대 결과
5. 개발자 전달용 요약 문장

## 바꿔야 할 부분

- '[버그/이상 현상]'

## 후속 질문

- 이 내용을 이슈 티켓 등록용 문장으로 더 짧게 다시 써줘.
- 개발자가 바로 재현할 수 있게 단계 설명을 더 구체적으로 보완해줘.

## 수정 요청

- 모호한 표현은 줄이고 재현 단계가 더 분명하게 보이게 바꿔줘.
- 실제 결과와 기대 결과 차이가 더 잘 드러나게 정리해줘.

## 검수 요청

- 이 설명만으로도 개발자가 상황을 이해할 수 있는지 검수해줘.
- 빠진 핵심 정보가 있다면 짚어줘.

## 사용 팁

버그 설명은 원인 추측보다 '재현 가능성'과 '기대 결과 대비 실제 결과'가 더 중요합니다.

---

---

## UI 수정 요청을 더 구체적으로 하고 싶을 때

## 한 줄 설명

막연한 디자인 피드백을 실제 수정 가능한 UI 요청 문장으로 바꾸는 작업지시서입니다.

## 이런 상황에서 특히 잘 맞습니다

- “뭔가 어색한데” 수준에서 더 구체적인 요청이 필요할 때
- 디자이너나 개발자에게 UI 수정 방향을 전달해야 할 때
- 색감, 간격, 위계, CTA 등 어느 부분이 문제인지 잘 정리되지 않을 때

## 이 작업지시서로 기대할 수 있는 결과

- 현재 UI 문제 진단
- 수정 우선순위
- 실제 전달 가능한 UI 수정 요청 문장

## 언제 쓰는지

- 화면 초안 검토 후
- 구현물 개선 요청 단계

## 바로 복사할 본문

현재 UI를 보니 방향은 맞지만, 수정 요청을 더 구체적으로 정리하고 싶습니다.  
이번 화면의 목적은 [전환 / 정보 전달 / 탐색 / 신청 / 구매 유도]입니다.  
특히 사용자 입장에서 가장 중요한 요소는 [CTA / 정보 위계 / 가독성 / 신뢰감 / 모바일 사용성]입니다.

먼저 현재 UI에서 가장 먼저 손봐야 할 문제를 짚어주세요.  
그 다음 아래 형식으로 수정 요청 문장을 정리해 주세요.

1. 현재 문제 요약
2. 왜 문제가 되는지
3. 우선 수정해야 할 부분 3개
4. 개발자/디자이너에게 전달할 수 있는 실제 수정 요청 문장

답변은 취향보다 사용자 경험 관점에서 정리해 주세요.  
가능하면 “무엇을 어떻게 바꾸면 더 좋아지는지”가 바로 보이게 써주세요.

## 바뀌야 할 부분

- `[전환 / 정보 전달 / 탐색 / 신청 / 구매 유도]`
- `[CTA / 정보 위계 / 가독성 / 신뢰감 / 모바일 사용성]`

## 후속 질문

- 위 수정 요청 중 가장 효과가 큰 2개만 우선순위로 다시 정리해줘.

- 모바일 기준으로만 봤을 때 수정 요청 문장도 다시 써줘.

## 수정 요청

- 취향성 표현은 줄이고, 사용자 행동 기준으로 다시 써줘.
- 너무 추상적인 말 대신 실제 화면 요소 기준으로 바꿔줘.

## 검수 요청

- 이 수정 요청만 전달해도 상대가 실제로 손볼 수 있는 수준인지 검수해줘.
- 빠진 핵심 개선 포인트가 있다면 보완해줘.

## 사용 팁

좋은 UI 피드백은 "별로다"가 아니라 "어느 요소가 왜 문제인지, 그래서 무엇을 바꾸면 좋은지"가 함께 있어야 합니다.

---

# API 연동 작업을 설명하고 싶을 때

## 한 줄 설명

API 연동이 필요한 기능이나 흐름을 비개발자도 이해할 수 있으면서 개발자 전달용으로도 쓸 수 있게 정리하는 작업지시서입니다.

## 이럴 때 가장 먼저 꺼내 쓰세요

- 기능 요구사항은 정리됐는데 외부 서비스와 연결되는 지점을 설명하기 어려울 때
- 개발자 전달 문서 전에 '왜 이 연동이 필요한지'와 '어디서 호출되는지'를 분명히 하고 싶을 때

## 이 항목이 다른 항목과 다른 점

이 항목은 기능 자체를 정의하는 문서가 아니라, '연동 목적·호출 지점·예외 상황'을 사용자 흐름 기준으로 설명하는 API 맥락 정리용입니다.

## 이런 상황에서 특히 잘 맞습니다

- 어떤 API를 왜 붙이는지 설명이 애매할 때
- 외부 서비스와 내부 기능 연결 흐름을 정리해야 할 때
- 기획자나 운영자가 개발자에게 연동 맥락을 전달해야 할 때

## 이 작업지시서로 기대할 수 있는 결과

- 연동 목적
- 사용자 흐름 안에서 API가 필요한 지점
- 요청/응답 관점의 설명
- 개발자 전달용 정리본

## 언제 쓰는지

- 기능 정의 이후 연동 범위를 설명하는 단계
- 개발자 전달 전 연동 맥락을 정리할 때

## 바로 복사할 본문

지금 저는 [서비스/기능]에 [API 또는 외부 서비스]를 연동하는 작업을 설명하고 싶습니다.

가장 중요한 것은 기술 용어를 늘어놓는 것이 아니라, 왜 이 연동이 필요한지와 어떤 흐름에서 쓰이는지를 분명히 정리하는 것입니다.

현재 목적은 [데이터 조회 / 제출 처리 / 알림 발송 / 외부 결제 연동 / 자동 발송 / 상태 확인]입니다.

사용자 입장에서 가장 중요한 흐름은 [어떤 행동 후 어떤 결과가 보여야 하는지]입니다.

아래 기준으로 연동 설명을 정리해 주세요.

1. 왜 이 API 연동이 필요한지
2. 사용자 흐름 안에서 어디서 호출되는지
3. 어떤 데이터가 오가야 하는지
4. 실패 시 어떤 예외 상황이 있는지
5. 개발자에게 전달 가능한 설명 요약

답변은 아래 순서로 주세요.

1. 연동 목적 요약
2. 사용자 흐름 기준 설명
3. 요청/응답 관점 핵심 포인트
4. 개발자 전달용 요약 문장

## 바뀌야 할 부분

- `[서비스/기능]`
- `[API 또는 외부 서비스]`
- `[데이터 조회 / 제출 처리 / 알림 발송 / 외부 결제 연동 / 자동 발송 / 상태 확인]`
- `[어떤 행동 후 어떤 결과가 보여야 하는지]`

## 후속 질문

- 위 내용을 개발 티켓용 문서 형식으로 더 짧게 다시 정리해줘.
- 실패 시 예외 상황과 대응 방향을 더 자세히 보강해줘.

## 수정 요청

- 기술 설명보다 실제 사용자 흐름이 더 먼저 보이게 다시 써줘.
- 개발자가 오해할 수 있는 애매한 표현은 줄여줘.

## 검수 요청

- 이 설명만으로도 개발자가 연동 목적과 흐름을 이해할 수 있는지 검수해줘.
- 빠진 핵심 연동 포인트가 있다면 짚어줘.

## 사용 팁

API 연동 설명은 “어떤 기술을 쓸지”보다 “왜 필요하고, 어디서 쓰이며, 실패하면 어떻게 되는지”가 먼저 보여야 좋습니다.

---

---

# MVP 범위와 추후 확장 범위를 나누고 싶을 때

## 한 줄 설명

아이디어나 기능 목록이 너무 넓을 때, 지금 꼭 필요한 범위와 나중에 붙일 범위를 나눠서 더 현실적인 계획으로 정리하는 작업지시서입니다.

## 이런 상황에서 특히 잘 맞습니다

- 만들고 싶은 기능이 많은데 우선순위가 안 잡힐 때
- 개발 범위가 계속 커지는 느낌일 때
- 서비스 초기 버전을 더 작고 선명하게 정리하고 싶을 때

## 이 작업지시서로 기대할 수 있는 결과

- MVP 범위
- 제외하거나 뒤로 미룰 항목
- 확장 후보 목록
- 우선 구현 이유

## 언제 쓰는지

- 기능 정의 후
- 개발 범위 조정 단계

## 바로 복사할 본문

지금 저는 [서비스/기능/프로젝트]의 구현 범위를 정리하고 싶습니다.  
아이디어는 많지만, 지금 바로 필요한 MVP 범위와 나중에 붙여도 되는 확장 범위를 나누는 것이 필요합니다.

현재 가장 중요한 목적은 [빠른 검증 / 전환율 확인 / 운영 단순화 / 핵심 가치 검증]입니다.  
따라서 아래 기준으로 범위를 나눠 주세요.

1. 지금 꼭 넣어야 하는 MVP 항목
2. 지금은 제외해도 되는 항목
3. 추후 확장 후보
4. 왜 그렇게 나누는지 이유
5. 범위를 더 작게 잡아야 하는 리스크가 있다면 무엇인지

답변은 아래 순서로 주세요.

1. 핵심 가치 기준 요약
2. MVP 포함 항목
3. MVP 제외 항목
4. 추후 확장 항목
5. 최종 추천 범위

## 바뀌야 할 부분

- `[서비스/기능/프로젝트]`
- `[빠른 검증 / 전환율 확인 / 운영 단순화 / 핵심 가치 검증]`

## 후속 질문

- 위 범위를 더 작게 줄인 초소형 MVP 버전도 다시 제안해줘.
- 반대로 조금 더 여유 있는 2차 범위도 정리해줘.

## 수정 요청

- 구현 난이도보다 검증 가치가 더 잘 보이게 다시 정리해줘.
- 제외 항목이 왜 제외되는지 더 선명하게 써줘.

## 검수 요청

- 이 범위가 실제로 MVP라고 부를 수 있는 수준인지 검수해줘.
- 너무 욕심낸 부분이 있다면 짚어줘.

## 사용 팁

MVP는 “적게 만드는 것”이 아니라 “핵심 가치를 가장 빨리 검증하는 범위”를 만드는 일입니다.

---

---

## 코드를 직접 짜기 전 구조 제안을 받고 싶을 때

### 한 줄 설명

구현 전에 컴포넌트, 파일, 페이지, 데이터 흐름 같은 구조를 먼저 제안받아 시행착오를 줄이는 작업지시서입니다.

### 이럴 때 가장 먼저 꺼내 쓰세요

- 기능 정의는 끝났지만 실제 코드를 어떤 구조로 나눌지 감이 없을 때
- 개발자 전달용 작업 지시문보다 한 단계 앞에서, '설계 뼈대'를 먼저 보고 싶을 때

### 이 항목이 다른 항목과 다른 점

이 항목은 기능 정의나 개발 전달 문서가 아니라, '파일/컴포넌트/상태 구조'를 미리 잡아 시행착오를 줄이는 설계 제안용 작업지시서입니다.

### 이런 상황에서 특히 잘 맞습니다

- 바로 코드를 쓰기보다 구조를 먼저 잡고 싶을 때
- 폴더/컴포넌트 분리가 애매할 때
- 나중에 확장 가능한 설계를 먼저 보고 싶을 때

### 이 작업지시서로 기대할 수 있는 결과

- 구조 제안
- 파일 분리 기준
- 페이지/컴포넌트 역할 정리
- 확장 시 유리한 설계 포인트

### 언제 쓰는지

- 구현 직전
- 구조 설계/리뷰 단계

### 바로 복사할 본문

지금 저는 [서비스/기능/화면]을 구현하려고 하지만, 코드를 바로 짜기 전에 구조를 먼저 정리하고 싶습니다.

가장 중요한 것은 [유지보수성 / 단순한 구조 / 재사용성 / 확장성 / 명확한 역할 분리]입니다.  
이번 구현에서 예상되는 요소는 [페이지 / 컴포넌트 / 데이터 / 상태 / 유틸]입니다.

아래 기준으로 구조를 제안해 주세요.

1. 파일/폴더를 어떻게 나누면 좋은지
2. 어떤 컴포넌트가 필요할지
3. 어떤 부분은 분리하고 어떤 부분은 단순하게 유지할지
4. 지금 단계에서 과도한 구조화가 될 위험은 없는지

답변은 아래 순서로 주세요.

1. 전체 구조 제안
2. 파일/폴더 예시
3. 역할 분리 기준
4. 구현 전 주의할 점

## 바뀌야 할 부분

- `[서비스/기능/화면]`
- `[유지보수성 / 단순한 구조 / 재사용성 / 확장성 / 명확한 역할 분리]`
- `[페이지 / 컴포넌트 / 데이터 / 상태 / 유틸]`

## 후속 질문

- 위 구조를 MVP 기준으로 더 단순하게 줄여줘.
- 나중에 확장될 수 있는 부분만 따로 표시해줘.

## 수정 요청

- 과한 구조화는 줄이고 현실적인 분리 기준으로 다시 써줘.
- 초보 개발자도 이해하기 쉽게 다시 정리해줘.

## 검수 요청

- 이 구조가 지금 단계에 과한지 적절한지 검수해줘.
- 빠진 핵심 구성 요소가 있다면 짚어줘.

## 사용 팁

구조 제안은 "멋진 아키텍처"보다 "지금 단계에서 덜 꼬이게 만드는 구조"가 더 중요합니다.

---

## 이미 나온 코드나 구현안을 개선 요청하고 싶을 때

### 한 줄 설명



이미 나온 코드나 구현안에서 실제로 더 나아질 부분을 짚고, 더 좋은 방향으로 개선 요청하는 구조 개선형 작업지시서입니다.

## 이런 상황에서 특히 잘 맞습니다

- 이미 코드 초안이 나왔는데 더 좋아질 여지가 클 때
- 구현 방향은 맞지만 구조가 아쉽거나 깔끔하지 않을 때
- AI나 개발자에게 “이 부분을 더 낮게 다듬어 달라”고 요청하고 싶을 때

## 이 작업지시서로 기대할 수 있는 결과

- 우선 개선 포인트
- 유지해도 되는 부분
- 구조적 애매함 진단
- 개선된 코드/구현안

## 언제 쓰는지

- 1차 구현 후 개선 라운드
- 코드가 돌아가지만 만족스럽지 않을 때

## 바로 복사할 본문

아래 코드는/구현안은 기본 동작은 되지만, 아직 완성도가 부족합니다.  
완전히 새로 만들기보다 현재 구조를 바탕으로 어떤 부분을 개선해야 하는지 먼저 짚어주세요.

이번 개선의 우선순위는 [가독성 / 구조 정리 / 재사용성 / 사용자 흐름 / 성능 / 문구 품질]입니다.

먼저 아래 순서로 점검해 주세요.

1. 지금 상태에서 가장 먼저 개선해야 할 부분
2. 유지해도 되는 부분
3. 구조적으로 애매한 부분
4. 사용자가 느낄 수 있는 문제

그 다음 아래 방식으로 답변해 주세요.

1. 개선 포인트 요약
2. 왜 고쳐야 하는지 설명
3. 더 나은 수정안 또는 개선된 코드/구현안

가능하면 단순 취향 차이보다, 실제로 더 좋아지는 방향 위주로 정리해 주세요.  
필요하면 개선 우선순위까지 함께 제안해 주세요.

## 바꿔야 할 부분

- “[가독성 / 구조 정리 / 재사용성 / 사용자 흐름 / 성능 / 문구 품질]”: 우선순위 선택

- 아래에 실제 코드나 구현안 붙여 넣기

## 후속 질문

- 지금 개선안에서 가장 효과가 큰 3가지만 우선순위로 다시 정리해줘.
- 코드 수정안 말고, 개발자에게 전달할 개선 요청 문구로도 다시 만들어줘.

## 수정 요청

- 너무 취향적인 지적은 줄이고 실제 영향이 큰 부분만 남겨줘.
- 개선 이유를 더 실무적으로 이해되게 설명해줘.

## 검수 요청

- 이 개선안이 실제로 더 나은 방향인지 검수해줘.
- 빠진 리스크나 놓친 개선 포인트가 있다면 보완해줘.

## 자주 하는 실수

- 코드 원문 없이 요약만 붙여 넣는다
- 무엇을 우선 개선할지 정하지 않는다
- “예쁘게 정리”와 “실제로 좋아짐”을 혼동한다

## 이 순서에서 쓰면 좋습니다

1. 기능 정의형 또는 전달형 작업지시서로 구현을 만든다
2. 이 작업지시서로 개선 포인트를 잡는다
3. 검수형으로 리스크를 마지막 점검한다

## 사용 팁

개선 요청은 “전부 바꿔줘”보다 “뭐부터 고치면 가장 좋아지는지”를 묻는 방식이 훨씬 유용합니다.

---

## 개발 결과물을 최종 검수하고 싶을 때

### 한 줄 설명

이미 구현된 화면이나 기능 결과물을 출시/공유 직전에 점검해서, 사용자 흐름과 완성도 관점의 마지막 리스크를 찾는 작업지시서입니다.

## 이럴 때 가장 먼저 꺼내 쓰세요

- 기능은 다 만들었는데 배포 전 `실제 사용 리스크`를 마지막으로 걸러내고 싶을 때
- 구조 제안이나 기능 정의는 끝났고, 지금은 결과물을 출고 가능한 상태인지 보고 싶을 때

## 이 항목이 다른 항목과 다른 점

이 항목은 구조 제안이나 API 설명이 아니라, `구현이 끝난 결과물`을 사용자 흐름·예외 처리·배포 리스크 관점에서 검수하는 최종 배포 전 점검용입니다.

## 이런 상황에서 특히 잘 맞습니다

- 기능은 구현됐지만 그대로 배포해도 될지 불안할 때
- 사용자 흐름 기준으로 빠진 부분이 없는지 보고 싶을 때
- 겉보기보다 실제 사용 시 문제가 없는지 점검하고 싶을 때

## 이 작업지시서로 기대할 수 있는 결과

- 최종 리스크 요약
- 사용자 관점의 문제 지점
- 고치면 좋은 우선순위
- 배포 전 최종 점검 포인트

## 언제 쓰는지

- 구현 완료 후
- 테스트/배포 직전 검수 단계

## 바로 복사할 본문

현재 [기능/화면/서비스] 구현은 거의 완료되었고, 배포 또는 공유 전에 최종 검수를 받고 싶습니다.

가장 중요한 것은 [사용자 흐름 / 가독성 / CTA / 오류 처리 / 완성도 / MVP 범위 적합성]입니다.  
이번 검수는 보기 좋은지보다, 실제 사용자 입장에서 문제 없이 쓰일 수 있는지에 초점을 맞춰주세요.

아래 기준으로 점검해 주세요.

1. 사용자 흐름이 자연스러운지
2. 빠진 핵심 상태나 예외 처리가 있는지
3. 문구나 UI가 오해를 부르는 부분이 있는지
4. MVP 범위를 넘거나 부족한 부분이 있는지
5. 배포 전 가장 먼저 고치면 좋은 부분이 무엇인지

답변은 아래 순서로 주세요.

1. 전체 평가 요약
2. 꼭 수정해야 할 부분

- 3. 수정하면 더 좋아지는 부분
- 4. 지금 배포해도 되는지 짧은 판단

## 바뀌야 할 부분

- `[기능/화면/서비스]`
- `[사용자 흐름 / 가독성 / CTA / 오류 처리 / 완성도 / MVP 범위 적합성]`

## 후속 질문

- 위 검수 결과 중 가장 치명적인 3가지만 우선순위로 다시 정리해줘.
- 지금 기준으로선 괜찮은 부분도 같이 짚어줘.

## 수정 요청

- 취향 차이보다 실제 사용자 영향이 큰 부분 중심으로 다시 검수해줘.
- 배포 전 체크 항목처럼 더 분명한 문장으로 바꿔줘.

## 검수 요청

- 이 상태가 정말 배포 가능한 수준인지 한 번 더 검수해줘.
- 빠진 예외 처리나 사용자 혼란 요소가 있다면 보완해줘.

## 사용 팁

최종 검수는 “예쁘다”보다 “실제로 쓰다 막히지 않는다”를 확인하는 단계입니다.

---

## 보조 작업지시서

아래 보조 작업지시서는 핵심 작업지시서를 더 오래, 더 깊게 쓰게 도와주는 연결 세트입니다.

## 빠진 핵심 요소가 있는지 점검하고 싶을 때

### 한 줄 설명

이미 나온 결과물에서 빠진 핵심 요소를 빠르게 찾아내고 보완 방향을 잡는 점검형 작업지시서입니다.

### 이럴 때 가장 먼저 꺼내 쓰세요

- 결과물을 통째로 다시 쓰기 전에 ‘무엇이 빠졌는지’부터 알고 싶을 때

- 최종 검수까지 가기 전, 누락 여부만 빠르게 선별하고 싶을 때

## 이 항목이 다른 항목과 다른 점

이 항목은 전체 완성도를 평가하는 최종 검수와 다르게, `빠진 요소`와 `약하게 들어간 요소`를 먼저 찾아내는 누락 탐지용 작업지시서입니다.

## 이런 상황에서 특히 잘 맞습니다

- 결과물은 있는데 뭔가 허전할 때
- 큰 방향은 맞지만 빠진 요소가 있을까 불안할 때
- 제출 전에 누락 여부를 점검하고 싶을 때

## 이 작업지시서로 기대할 수 있는 결과

- 빠진 핵심 요소 목록
- 보완 우선순위
- 누락 반영 방향

## 언제 쓰는지

- 초안 이후 점검 단계
- 제출 전 체크 단계

## 바로 복사할 본문

아래 결과물은 전체 방향은 맞지만, 혹시 빠진 핵심 요소가 없는지 점검하고 싶습니다.

이 결과물의 목적은 [보고 / 공유 / 게시 / 제안 / 전달]입니다.  
읽는 사람은 [상사 / 고객 / 사용자 / 팀원 / 일반 독자]입니다.

아래 기준으로 점검해 주세요.

1. 빠진 핵심 요소가 있는지
2. 있어야 하는데 약하게 들어간 요소가 있는지
3. 지금 보완하면 가장 좋아지는 부분이 무엇인지

답변은 아래 순서로 주세요.

1. 누락 또는 약한 요소 요약
2. 우선 보완할 3가지
3. 보완 방향 또는 예시 문장

## 바꿔야 할 부분

- `[보고 / 공유 / 게시 / 제안 / 전달]`
- `[상사 / 고객 / 사용자 / 팀원 / 일반 독자]`

## 후속 질문

- 위에서 가장 중요한 1가지만 먼저 보완한 버전으로 다시 써줘.
- 빠진 요소를 반영한 최종본도 하나 만들어줘.

## 수정 요청

- 사소한 표현보다 실제로 중요한 누락 요소 중심으로 정리해줘.
- 보완 방향을 더 실전적으로 설명해줘.

## 검수 요청

- 지금 상태에서 정말 빠진 핵심 요소가 있는지 한 번 더 검수해줘.
- 보완 우선순위가 적절한지도 점검해줘.

## 사용 팁

결과물이 약할 때는 전체를 새로 쓰기보다, 빠진 요소 1~2개를 보완하는 쪽이 훨씬 빠를 때가 많습니다.

---

## 수정 요청을 어떻게 이어갈지 설계하고 싶을 때

### 한 줄 설명

첫 결과가 나온 뒤 어떤 순서와 기준으로 수정 요청을 이어가야 할지 설계하는 수정 흐름형 작업지시서입니다.

### 단계 태그

‘수정’

### 이럴 때 가장 먼저 꺼내 쓰세요

- 첫 결과는 나왔지만 어떻게 고쳐달라고 해야 할지 매번 막연할 때
- 단일 수정 요청이 아니라 ‘수정의 순서’와 ‘판단 기준’까지 정리하고 싶을 때

### 이 항목이 다른 항목과 다른 점

이 항목은 후속 질문 흐름처럼 더 묻는 단계가 아니라, ‘나온 결과를 어떤 순서로 고쳐가야 하는지’를 설계하는 수정 단계 전용 작업지시서입니다.

## 이런 상황에서 특히 잘 맞습니다

- 첫 결과는 나왔는데 어떻게 고쳐달라고 해야 할지 모르겠을 때
- 수정 요청이 늘 막연하거나 반복될 때
- 결과를 끝까지 다듬는 흐름이 필요할 때

## 이 작업지시서로 기대할 수 있는 결과

- 수정 요청 단계 설계
- 어떤 상황에서 어떤 수정 요청을 해야 하는지 기준
- 바로 복사해서 쓸 수정 요청 문장

## 언제 쓰는지

- 첫 답변 이후 보완 단계
- 결과 완성도 끌어올리기 단계

## 바로 복사할 본문

지금 저는 [작업 주제]에 대해 첫 결과를 받은 상태입니다.

문제는 결과가 완벽하지 않은데, 어떤 순서로 수정 요청을 이어가야 할지 모르겠다는 점입니다.

현재 가장 자주 부족한 부분은 [너무 일반적임 / 구조가 약함 / 톤이 안 맞음 / 예시가 부족함 / 전달력이 약함]입니다.

아래 기준으로 수정 요청 흐름을 설계해 주세요.

1. 어떤 부분을 먼저 고쳐야 하는지
2. 어떤 상황에서 어떤 수정 요청 문장을 써야 하는지
3. 수정 요청을 몇 단계로 나누면 좋은지
4. 마지막에는 어떤 기준으로 마무리 검수를 해야 하는지

답변은 아래 순서로 주세요.

1. 수정 요청 4단계 흐름
2. 각 단계의 목적
3. 바로 복사해서 쓸 수정 요청 문장

## 바뀌야 할 부분

- '[작업 주제]'
- '[너무 일반적임 / 구조가 약함 / 톤이 안 맞음 / 예시가 부족함 / 전달력이 약함]'

## 후속 질문

- 위 흐름을 더 짧고 실전형으로 다시 줄여줘.
- 초보자 기준으로 더 쉽게 따라갈 수 있게 다시 정리해줘.

## 수정 요청

- 설명형 부분은 줄이고 수정 요청 문장 자체를 더 강화해줘.
- 단계 간 차이가 더 잘 보이게 다시 정리해줘.

## 검수 요청

- 이 흐름만 따라가도 결과를 충분히 다듬을 수 있는지 검수해줘.
- 빠진 핵심 수정 단계가 있다면 보완해줘.

## 사용 팁

수정 요청은 "다시 써줘"보다 "무엇을, 어떤 기준으로, 어떤 순서로 고칠지"가 분명할수록 훨씬 강해집니다.

---

## 검수 질문을 어떻게 던질지 정리하고 싶을 때

### 한 줄 설명

결과물을 마지막으로 점검할 때 어떤 질문을 던져야 품질이 좋아지는지, 검수 질문 자체를 설계하는 작업지시서입니다.

### 단계 태그

‘검수’

### 이럴 때 가장 먼저 꺼내 쓰세요

- 결과물은 거의 끝났지만 어떤 질문으로 마지막 점검을 해야 할지 막막할 때
- "한 번 더 봐줘"가 아니라 실제 판단 가능한 검수 질문 세트를 만들고 싶을 때

### 이 항목이 다른 항목과 다른 점

이 항목은 결과물을 직접 고치는 수정 단계가 아니라, '무엇을 질문해야 품질 문제가 드러나는지'를 설계하는 검수 단계 전용 작업지시서입니다.

### 이런 상황에서 특히 잘 맞습니다

- 마지막 검수를 매번 막연하게 하게 될 때
- "한 번 더 봐줘"보다 더 좋은 질문이 필요할 때
- 결과물 종류에 맞게 검수 질문을 잘 바꾸고 싶을 때



## 이 작업지시서로 기대할 수 있는 결과

- 검수 질문 구조
- 목적별 검수 질문
- 바로 복사해서 쓸 최종 검수 질문 문장

## 언제 쓰는지

- 결과물 마무리 전
- 최종 점검 루틴 만들기 단계

## 바로 복사할 본문

지금 저는 [결과물 종류]를 거의 마무리한 상태이며, 마지막 검수 질문을 더 잘 던지고 싶습니다.

검수의 목적은 [빠진 요소 확인 / 읽기 쉬움 확인 / 설득력 확인 / 제출 가능 여부 판단 / 오류 점검]입니다.  
읽는 사람은 [상사 / 고객 / 사용자 / 팀원 / 일반 독자]입니다.

아래 기준으로 검수 질문을 설계해 주세요.

1. 꼭 확인해야 할 핵심 질문
2. 결과물 종류에 맞는 검수 질문
3. 너무 막연하지 않고 실제로 도움이 되는 질문
4. 최종 제출 여부를 판단하는 질문

답변은 아래 순서로 주세요.

1. 검수 질문 5개
2. 그중 가장 중요한 핵심 질문 1개
3. 바로 복사해서 쓸 최종 검수 질문 문장

## 바뀌야 할 부분

- '[결과물 종류]'
- '[빠진 요소 확인 / 읽기 쉬움 확인 / 설득력 확인 / 제출 가능 여부 판단 / 오류 점검]'
- '[상사 / 고객 / 사용자 / 팀원 / 일반 독자]'

## 후속 질문

- 위 검수 질문을 더 짧고 실전형으로 다시 줄여줘.
- 초보자도 바로 쓸 수 있게 설명을 더 쉽게 붙여줘.

## 수정 요청

- 막연한 질문은 줄이고 실제로 판단 가능한 질문 중심으로 바꿔줘.
- 결과물 종류에 더 맞는 질문으로 다시 다듬어줘.

## 검수 요청

- 이 검수 질문 세트가 실제 품질 점검에 도움이 되는지 검수해줘.
- 빠진 핵심 검수 관점이 있다면 보완해줘.

## 사용 팁

좋은 검수 질문은 “어때?”가 아니라 “무엇이 빠졌는지, 무엇이 약한지, 지금 내보내도 되는지”를 분명히 묻는 질문입니다.

---

---

## 같은 작업을 다른 AI 도구에 맞게 바꾸고 싶을 때

### 한 줄 설명

같은 목적의 작업지시서를 ChatGPT, Claude, Gemini, Codex 등 다른 도구에 넣어도 잘 맞게 조정하는 작업지시서입니다.

### 단계 태그

‘변형’

### 이럴 때 가장 먼저 꺼내 쓰세요

- 이미 잘 쓰고 있는 작업지시서가 있는데 다른 AI에도 맞게 옮겨 쓰고 싶을 때
- 결과 비교 실험을 하거나, 특정 도구에서 더 나은 반응을 끌어내고 싶을 때

### 이 항목이 다른 항목과 다른 점

이 항목은 첫 질문을 만드는 단계가 아니라, ‘이미 있는 작업지시서’를 다른 AI의 응답 성향에 맞춰 재배치하는 변형용 작업지시서입니다.

### 이런 상황에서 특히 잘 맞습니다

- 같은 작업을 여러 AI에서 시험해보고 싶을 때
- 도구마다 결과 톤이나 응답 방식이 달라서 보정이 필요할 때
- 한 작업지시서를 여러 도구에 맞게 재활용하고 싶을 때

### 이 작업지시서로 기대할 수 있는 결과

- 도구별 문장 조정 포인트

- 같은 목적을 유지한 변형 버전
- 여러 AI에서 재활용 가능한 입력문

## 언제 쓰는지

- 기존 작업지시서를 다른 AI에도 써보고 싶을 때
- 결과 비교 실험 단계

## 바로 복사할 본문

지금 저는 아래 작업지시서를 [현재 AI 도구] 기준으로 쓰고 있습니다.  
이제 같은 목적을 유지한 채 [다른 AI 도구]에도 더 잘 맞게 바꾸고 싶습니다.

가장 중요한 것은 목적은 유지하되, 도구 특성에 따라 문장 구조와 요청 방식을 조금 더 맞추는 것입니다.

아래 기준으로 작업지시서를 다시 정리해 주세요.

1. 현재 문장에서 그대로 유지해도 되는 부분
2. 다른 AI 도구에 맞게 바꾸면 좋은 부분
3. 더 잘 작동할 가능성이 높은 문장 구조
4. 최종 변형본

답변은 아래 순서로 주세요.

1. 현재 버전과 다른 도구의 차이를 고려한 조정 포인트
2. 수정 방향
3. 최종 변형 작업지시서

## 바뀌야 할 부분

- `[현재 AI 도구]`
- `[다른 AI 도구]`
- 아래에 현재 작업지시서 원문 붙여 넣기

## 후속 질문

- 위 변형본을 더 짧은 버전으로도 다시 만들어줘.
- 각 도구별로 조금씩 다르게 조정한 버전 2개를 추가해줘.

## 수정 요청

- 도구 차이는 반영하되 목적이 흔들리지 않게 다시 다듬어줘.
- 표현 차이보다 실제 결과 품질 차이가 날 만한 부분을 더 강조해줘.

## 검수 요청

- 이 변형본이 다른 AI 도구에서도 잘 작동할 가능성이 높은지 검수해줘.

- 여전히 원래 도구에만 최적화된 표현이 남아 있다면 짚어줘.

## 사용 팁

같은 작업이라도 AI마다 결과를 받는 방식이 조금 다를 수 있습니다.

완전히 새로 쓰기보다, 목적은 유지한 채 문장 구조와 요청 밀도만 조정하는 쪽이 더 효율적입니다.

---

## 전체 작업 흐름을 한 번에 설계하고 싶을 때

### 한 줄 설명

첫 질문부터 후속 질문, 수정 요청, 검수 질문까지 하나의 결과를 끝까지 완성하기 위한 전체 작업 흐름 설계형 작업지시서입니다.

### 이런 상황에서 특히 잘 맞습니다

- 단발성 질문이 아니라 결과물을 끝까지 완성하고 싶을 때
- 처음부터 전체 흐름을 설계해서 덜 헤매고 싶을 때
- AI 활용을 더 체계적으로 해보고 싶을 때

### 이 작업지시서로 기대할 수 있는 결과

- 첫 질문
- 후속 질문
- 수정 요청 질문
- 추가 고도화 질문
- 마지막 품질 검수 질문

### 언제 쓰는지

- 작업 시작 전
- “이번에는 처음부터 끝까지 제대로 해보고 싶다”는 시점

### 바로 복사할 본문

지금 저는 [목표 작업]을 AI와 함께 완성하고 싶습니다.

한 번 질문하고 끝내는 것이 아니라, 처음부터 결과물을 더 잘 만들 수 있도록 전체 작업 흐름을 설계해 주세요.

현재 제가 알고 있는 상황은 다음과 같습니다.

- 목적: [무엇을 만들고 싶은지]

- 대상: [누가 보거나 쓰는지]
- 현재 막힌 부분: [어디서 막히는지]
- 원하는 최종 결과: [어떤 상태를 원하는지]

먼저 이 작업을 시작하기 전에 꼭 확인해야 할 질문이 있다면 먼저 제시해 주세요.  
그 다음 아래 순서로 전체 흐름을 설계해 주세요.

1. 첫 질문
2. 첫 결과를 받은 뒤 이어갈 후속 질문
3. 결과가 약할 때 수정 요청하는 질문
4. 완성도를 높이기 위한 추가 질문
5. 마지막 품질 검수 질문

각 단계마다 왜 필요한지도 짧게 설명해 주세요.  
가능하면 각 단계마다 바로 복사해서 쓸 문장 예시도 함께 붙여 주세요.  
설명만 길게 하지 말고, 실제로 그대로 따라할 수 있는 흐름으로 정리해 주세요.

## 바뀌야 할 부분

- '[목표 작업]': 예) 블로그 글 완성, 보고서 작성, 기능 정의, 제안서 고도화
- '[무엇을 만들고 싶은지]': 실제 목적
- '[누가 보거나 쓰는지]': 대상
- '[어디서 막히는지]': 현재 문제
- '[어떤 상태를 원하는지]': 원하는 완성 결과

## 후속 질문

- 위 흐름을 더 짧은 실전형 버전으로 다시 줄여줘.
- 각 단계에서 실패하기 쉬운 포인트까지 같이 설명해줘.

## 수정 요청

- 설명보다 실제 복사할 문장 예시에 더 집중해서 다시 정리해줘.
- 단계 간 차이가 더 분명하게 보이도록 흐름을 다시 다듬어줘.

## 검수 요청

- 이 흐름만 따라가도 결과물을 끝까지 만들 수 있는지 검수해줘.
- 빠진 단계가 있다면 보완해줘.

## 자주 하는 실수

- 목적은 있는데 대상과 최종 결과가 불분명하다
- 수정 요청 단계를 빼먹는다

- 첫 질문만 정리하고 후속 흐름을 안 만든다

## 이 순서에서 쓰면 좋습니다

1. 이 작업지시서로 전체 흐름을 잡는다
2. 각 단계에 맞는 카테고리 작업지시서로 들어간다
3. 마지막에 검수형으로 마무리한다

## 사용 팁

이 작업지시서는 단일 작업지시서가 아니라 “AI를 쓰는 전체 방식”을 바꾸는 데 더 큰 가치가 있습니다. 결과물이 중요한 작업일수록, 처음부터 이 흐름을 잡아두는 편이 훨씬 강합니다.

---

## 마지막 메모

이 5개는 스타터팩 우선 제작 15개의 마지막 퍼즐입니다.

1단계는 시작점, 2단계는 확장과 정교화, 3단계는 ‘맞춤화’, ‘설득’, ‘개선’, ‘최종 판단’, ‘전체 흐름 설계’를 담당합니다.

이 묶음까지 올라오면 스타터팩 우선 제작 15개는 단순 초안을 넘어 실제 판매 기준선으로 볼 수 있습니다.

---

## 마지막 활용 팁

이 팩은 한 개 문장을 던지고 끝내는 자료가 아니라, 핵심 작업지시서와 보조 작업지시서를 조합해서 결과를 끝까지 다듬는 데 목적이 있습니다.

먼저 지금 가장 가까운 핵심 작업지시서 1개를 쓰고, 결과가 약하면 보조 작업지시서의 후속/수정/검수 흐름으로 이어가세요.